



Nombre y Apellido:

Legajo:

Examen Parcial 1

1. Dada la siguiente recurrencia:

$$\begin{aligned}T(1) &= 1 \\T(n) &= 4 * T(\lfloor n/2 \rfloor) + \lg n \quad \text{si } n > 1\end{aligned}$$

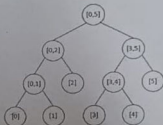
- Dar una cota asintótica Θ para T . No demuestre que la cota es correcta pero escriba brevemente cómo la halló.
- Mostrar usando el método de sustitución que dicha cota es una cota O de T .

2. Un árbol de segmentos es un árbol binario que almacena, además de los elementos de un arreglo, información sobre intervalos o segmentos del mismo. Esta información puede incluir por ejemplo la suma de los elementos de intervalos del arreglo o el menor elemento de un intervalo, permitiendo realizar consultas de rango sobre un arreglo de manera eficiente, por ejemplo, se puede encontrar la suma de elementos consecutivos de un arreglo en $O(\lg n)$, siendo n el largo del arreglo.

Implementaremos árboles de segmentos como árboles binarios, donde cada hoja o nodo almacena un intervalo y se cumplen los siguientes invariantes:

- Las hojas contienen intervalos unitarios.
- Los nodos contienen como intervalo el que se obtiene de unir el intervalo del hijo izquierdo con el del hijo derecho, los cuales deben ser intervalos consecutivos (dos intervalos (i, j) y (k, l) son consecutivos si $j + 1 = k$).

El siguiente es un ejemplo de un árbol de segmentos, que muestra solo la información de los intervalos:

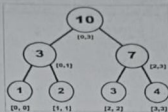


En Haskell utilizaremos el siguiente tipo de datos para representarlos:

```
type Interval = (Int, Int)
data STree a = Leaf Interval a | Node Interval a (STree a) (STree a)
```

donde el valor de tipo a contendrá el elemento del arreglo correspondiente a la posición dada por el intervalo unitario, en el caso de una hoja; o información asociada al segmento del arreglo, en el caso de un nodo. Definir las siguientes funciones de manera eficiente en Haskell.

- Definir una función $isSTree :: STree a \rightarrow \text{Bool}$, que determine si un árbol es un árbol de segmento. Dar la recurrencia correspondiente al trabajo de la función (no resolverla).
- De ahora en más supondremos que almacenaremos en los nodos de los árboles de segmentos la suma correspondiente al intervalo que representa el nodo. El siguiente es un ejemplo de un árbol de segmentos de este tipo para el arreglo $a = [1, 2, 3, 4]$.



Definir $\text{fromList} :: \text{Num } a \Rightarrow [a] \rightarrow \text{STree } a$, que construya un árbol de segmentos de la menor altura posible a partir de una lista. Por ejemplo $\text{fromList } [1, 2, 3, 4]$ debería devolver el árbol del ejemplo anterior.

- c) Definir $\text{nth} :: \text{Num } a \Rightarrow \text{Int} \rightarrow \text{STree } a \rightarrow \text{Maybe } a$, que permita acceder a los elementos del arreglo almacenado en un árbol de segmento a través de su posición en el arreglo. Resolver la función con costo $O(h)$, donde h es la altura del árbol.
- d) Definir $\text{included} :: \text{Interval} \rightarrow \text{Interval} \rightarrow \text{Bool}$, que determine si un intervalo está incluido o es igual a otro.
- e) Definir una función $\text{sumQuery} :: \text{Num } a \Rightarrow \text{STree } a \rightarrow \text{Interval} \rightarrow a$, que dado un árbol de segmento y un intervalo, devuelva la suma de los elementos del arreglo correspondiente al intervalo dado. Para ello usar la información almacenada en los nodos del árbol. Ayuda: La función del apartado anterior puede ser útil.